

Computer Science & IT

Database Management System



Transactions & concurrency control

Lecture No. 03

By- Vishal Sir



Recap of Previous Lecture



- ✓ Topic Isolation
- ✓ Topic Schedule
- ✓ Topic Equivalent schedule
- ✓ Topic Serializable schedule
- ✓ Topic Consistency

Topics to be Covered



✓ Topic

Equivalent schedule

✓ Topic

Serializable schedule

✓ Topic

RW problem

✓ Topic

WR problem

✓ Topic

WW problem

✓ Topic

Lost update problem



Topic : Isolation

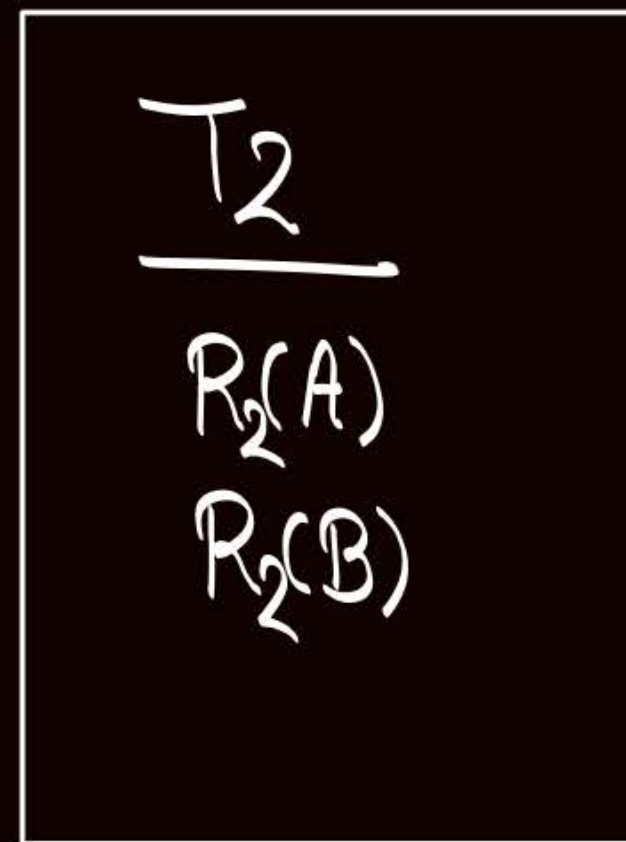
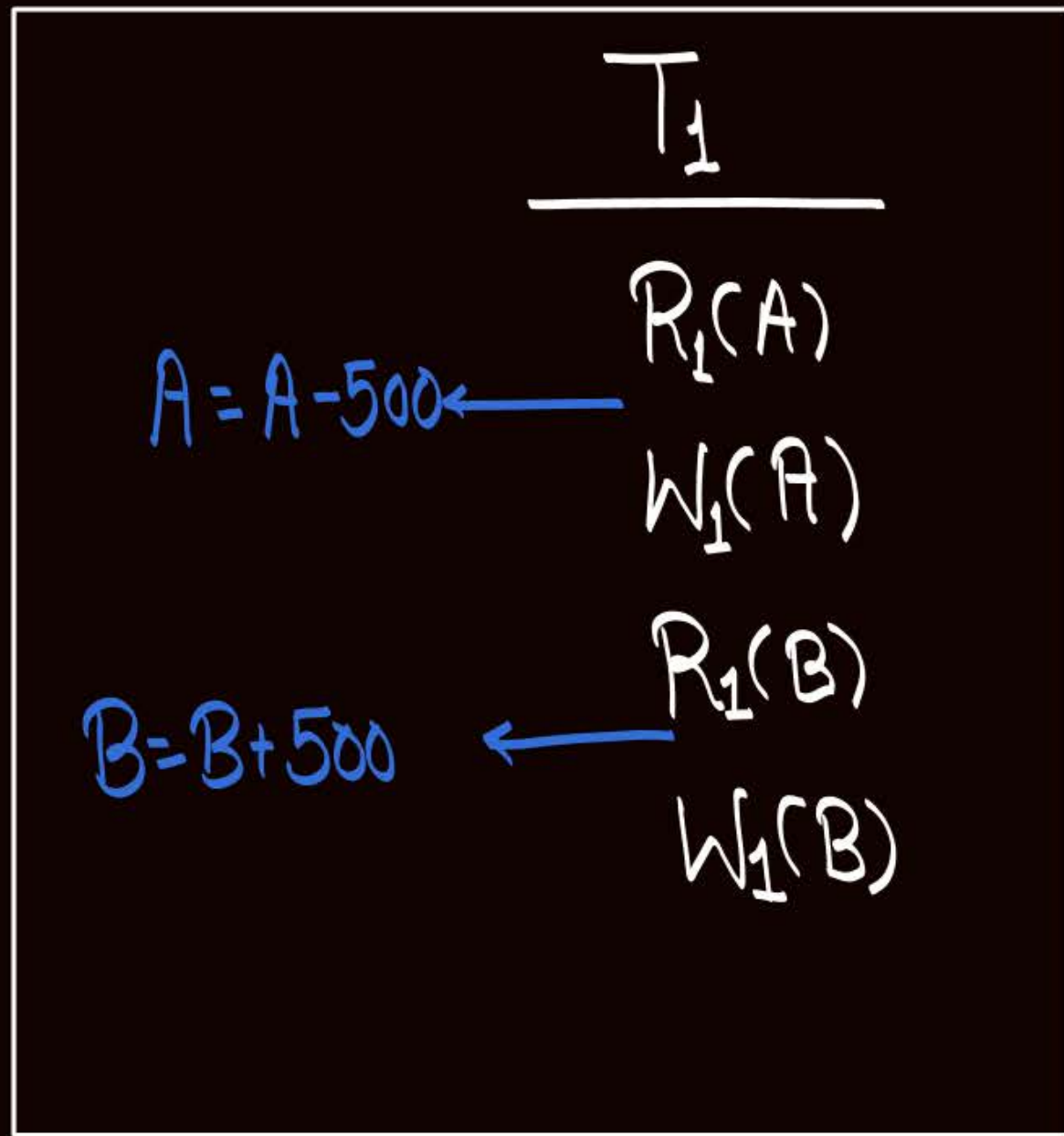


Isolation says that if two or more transactions are executing Concurrently, then they must be unaware of each other.

Consider two transactions.

T₁: Transfer Rs 500/- from account A to account B.

T₂: Read the current balance in account A & account B respectively



$R_i(x)$: It represent read of data item x by transaction T_i

$W_j(y)$: It represent Write of data item y by transaction T_j



Topic : Schedule

Time ordered sequence of operations of two or more transaction is called schedule.

eg:

S	
T ₁	T ₂
R ₁ (A)	
	R ₂ (A)
W ₁ (A)	
	R ₂ (B)
R ₁ (B)	
W ₁ (B)	

time
increasing
↓

$\equiv S = R_1(A); R_2(A); W_1(A); R_2(B); R_1(B); W_1(B);$

→ Consider the following schedule

$R_2(A); W_3(B); R_1(B); W_2(A); R_3(A); W_1(B); \equiv$

T_1	T_2	T_3
	$R_2(A)$	
		$W_3(B)$
$R_1(B)$		
	$W_2(A)$	
		$R_3(A)$
$W_1(B)$		



Topic : Schedule



There are two types of Schedules :

- ① Serial Schedule (i.e, one after another)
- ② Concurrent Schedule (i.e; interleaved execution)



Topic : Serial Schedule

- * If we can start the execution of a new transaction only when the previously started transaction has completed execution, then such schedules are called serial schedules.

eg. Consider two transactions
 $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$
 $T_2: R_2(A), R_2(B)$

If there are two transactions, then only two serial schedules are possible

(i) Execute T_1 , then execute T_2

$T_1 \xrightarrow{III} T_2$

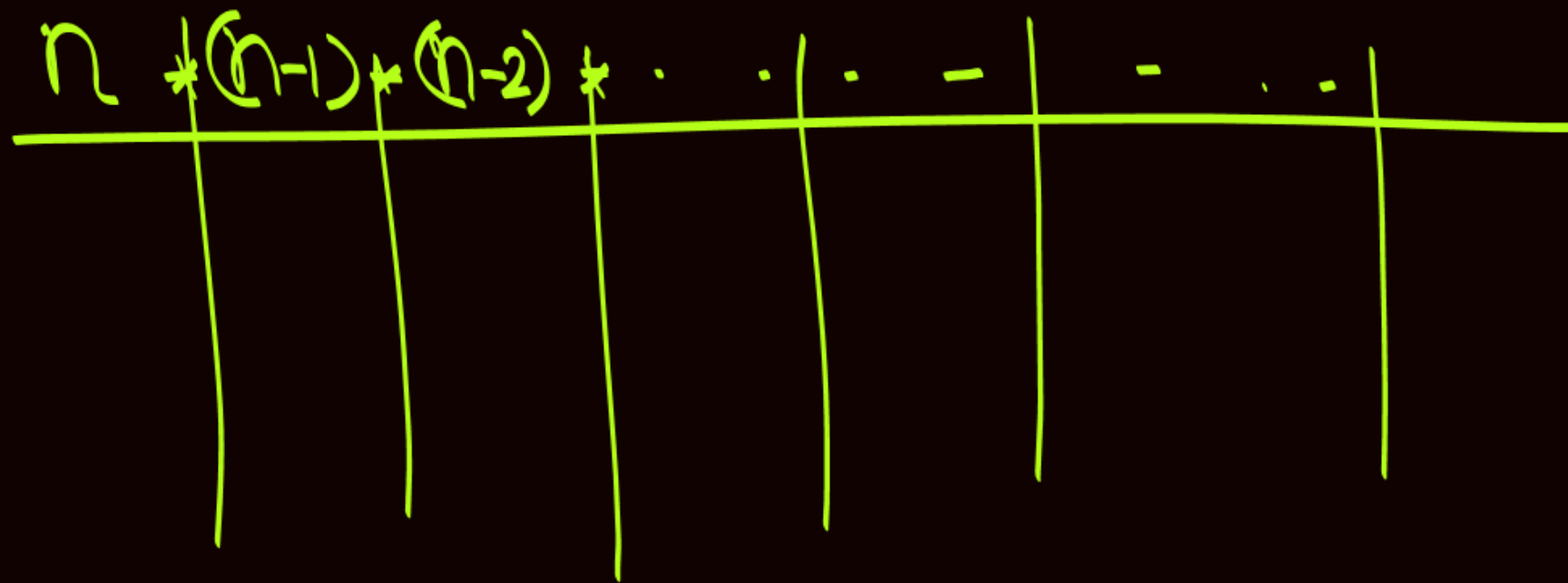
T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	
	$R_2(A)$
	$R_2(B)$

(ii) Execute T_2 , then execute T_1

$T_2 \xrightarrow{III} T_1$

T_1	T_2
	$R_2(A)$
	$R_2(B)$
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	

Note:- If there are 'n' transactions, then $n!$ serial schedules are possible over those transactions





Topic : Serial Schedule



* Serial schedule will always satisfy isolation Condition. (because the operations of transactions are not executing in interleaved manner)

But throughput of the system may be low with serial schedules.

↳ ∴ We think about Concurrent Schedules.



Topic : Concurrent Schedule

If operations of transactions can execute in interleaved manner, then such schedules are called Concurrent Schedules

eg. Consider two transactions
 $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

$T_2: R_2(A), R_2(B)$

and following schedules over transactions T_1 & T_2

S_1

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
	$R_2(B)$
$R_1(B)$	
$W_1(B)$	

S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

eg. Consider two transactions

$T_1: R_1(A), W_1(A), R_1(B), W_1(B)$ { is to transfer 500 from A to B }

$T_2: R_2(A), R_2(B)$ { Read balance in A & B }

let initially $A = 1000$
 $B = 0$

If there are two transactions, then only two serial schedules are possible

(i) Execute T_1 , then execute T_2

$T_1 \xrightarrow{|||} T_2$

T_1	T_2
$A = 1000 \leftarrow R_1(A)$	
$A = 500 \leftarrow W_1(A)$	
$B = 0 \leftarrow R_1(B)$	
$B = 500 \leftarrow W_1(B)$	
	$R_2(A) \text{ --- } A = 500$
	$R_2(B) \text{ --- } B = 500$

(ii) Execute T_2 , then execute T_1

$T_2 \xrightarrow{|||} T_1$

T_1	T_2
	$R_2(A) \text{ --- } A = 1000$
	$R_2(B) \text{ --- } B = 0$
$A = 1000 \text{ --- } R_1(A)$	
$A = 500 \text{ --- } W_1(A)$	
$B = 0 \text{ --- } R_1(B)$	
$B = 500 \text{ --- } W_1(B)$	

Note :- With concurrent schedule throughput of the system will increase, but isolation condition may not be satisfied

(S₁)

	T ₁	T ₂
A = 1000	R ₁ (A)	
A = 500	W ₁ (A)	
		R ₂ (A) — A = 500
		R ₂ (B) — B = 0
	R ₁ (B)	
	W ₁ (B)	

It is neither same as serial schedule T₁ → T₂ nor same as serial schedule T₂ → T₁

then T₂ will be able to identify that it is executing along with T₁

∴ isolation is not satisfied.

(S₂) ✓

	T ₁	T ₂
A = 1000	R ₁ (A)	
A = 500	W ₁ (A)	
		R ₂ (A) — 500
B = 0	R ₁ (B)	
B = 500	W ₁ (B)	
		R ₂ (B) — 500

All the values are exactly same as serial schedule T₁ → T₂

∴ None of the transaction will be able to identify that they are executing along with other transactions.

Hence, isolation is satisfied.

Note :-

In order to say that a schedule 'S' satisfies the isolation Condition, the behavior of schedule S must be equivalent to at least one of the serial schedule w.r.t. transaction of schedule S.



Topic : Isolation



- ▶ In order to satisfy the isolation condition, behaviour of schedule must be equivalent to at least one of the serial schedule.



Topic : Serializable schedule

A schedule is said to be serializable if and only if its behaviour is equivalent to at least one of the serial schedule.

- * Every serial schedule is a serializable schedule, but every serializable schedule need not be a serial schedule.



Topic : Isolation



- In order to satisfy the isolation Condition, behaviour of δ schedule must be equivalent to at least one of the serial δ schedule.

(or)

In order to satisfy the isolation Condition, the δ schedule must be a serializable δ schedule.

<u>S₁</u>	
T ₁	T ₂
A=1000 — R ₁ (A)	
A=500 — W ₁ (A)	
	R ₂ (A) — A=500
	R ₂ (B) — B=0
R ₁ (B)	
W ₁ (B)	

It is neither same as serial schedule T₁ → T₂ nor same as serial schedule T₂ → T₁

The behaviour of Schedule S₁ is not equivalent to any of the serial schedules over the transactions T₁ & T₂
 ∴ S₁ is not a serializable schedule
 Hence S₁ does not satisfy isolation

<u>S₂</u>	
T ₁	T ₂
A=1000 — R ₁ (A)	
A=500 — W ₁ (A)	
	R ₂ (A) — 500
B=0 — R ₁ (B)	
B=500 — W ₁ (B)	
	R ₂ (B) — 500

Behaviour of schedule S₂ is not equivalent to serial schedule T₂ → T₁
 i.e. S₂ ≠ T₂ → T₁
 But schedule S₂ is equivalent to serial schedule T₁ → T₂
 i.e. S₂ ≡ T₁ → T₂
 Schedule S₂ is equivalent to at least one of the serial schedules
 ∴ S₂ is a serializable schedule
 Hence S₂ satisfies isolation Condⁿ.

Note :- Concurrency Control Component of DBMS is responsible for avoiding the execution of non-serializable schedule.

⇓
i.e. Concurrency Control Component ensures serializability

⇓
i.e. Concurrency Control Component ensures isolation.



Topic : Consistency



Consistency states that before and after the execution of schedule the database must remain Consistent

• Criteria for Consistency:-

- ① Schedule must be strict recoverable schedule
- and ② Schedule must be serializable schedule

Note:

- ① For atomicity & durability, Recovery management Comes into the picture
- ② For isolation, Concurrency Control Component Comes into the picture
- ③ For Consistency, both recovery management Component and Concurrency Control Component Comes into the picture.

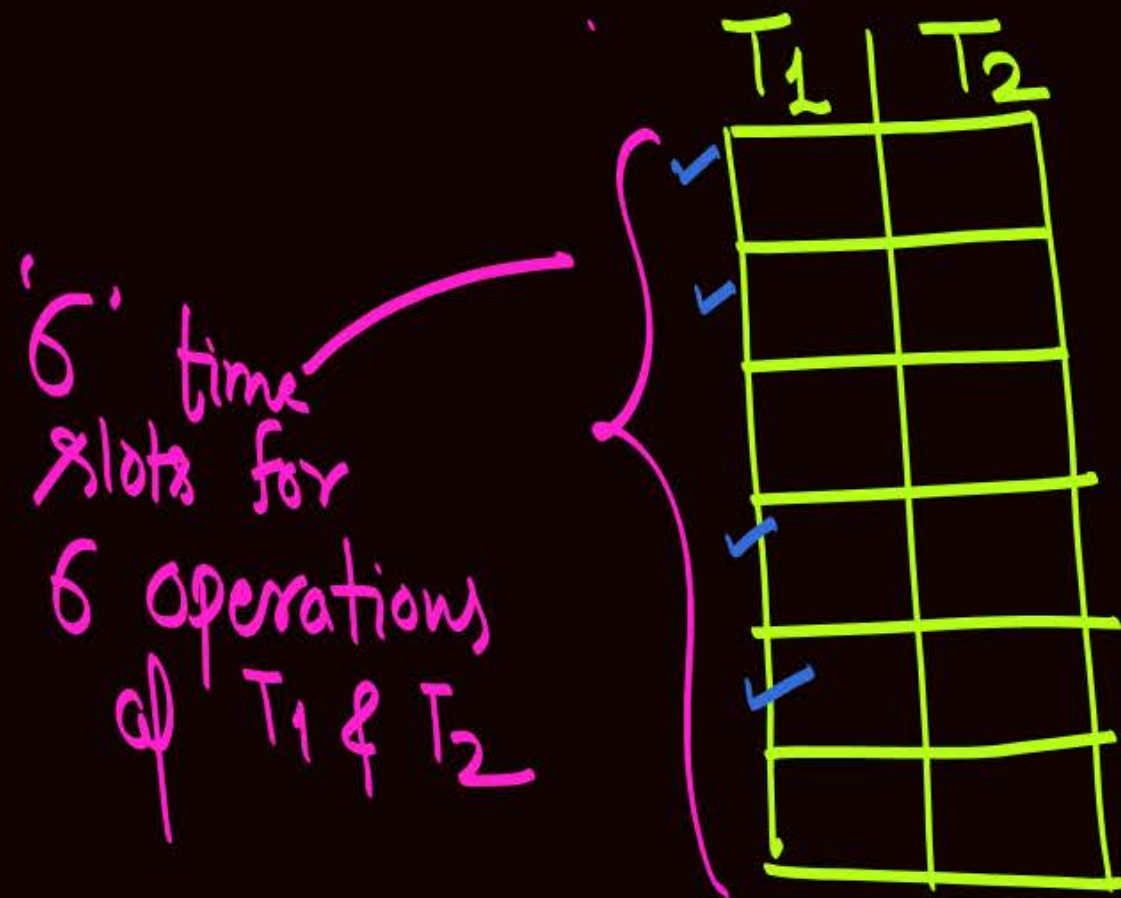
Home Work:-

Consider two transactions:-

✓ $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

✓ $T_2: R_2(A), R_2(B)$

How many total schedules are possible over the transactions T_1 & T_2 .



We can choose '4' time slots for '4' operations of transaction T_1 in 6C₄ ways and at those 4 positions '4' op of transaction T_1 can be arranged only in 1 way, because we can not change the order of op within the transaction.

$$\begin{aligned}\therefore \text{total no of schedules possible} &= \left({}^6C_4 * 1 \right) * \left({}^2C_2 * 1 \right) \\ &= {}^6C_4 * 1 * 1 * 1 = {}^6C_4 = \underline{15} \text{ Ans}\end{aligned}$$

$$\begin{aligned}&= \left({}^6C_2 * 1 \right) * \left({}^4C_4 * 1 \right) \\ &= {}^6C_2 * 1 * 1 * 1 = {}^6C_2 = \underline{15} \text{ Ans}\end{aligned}$$

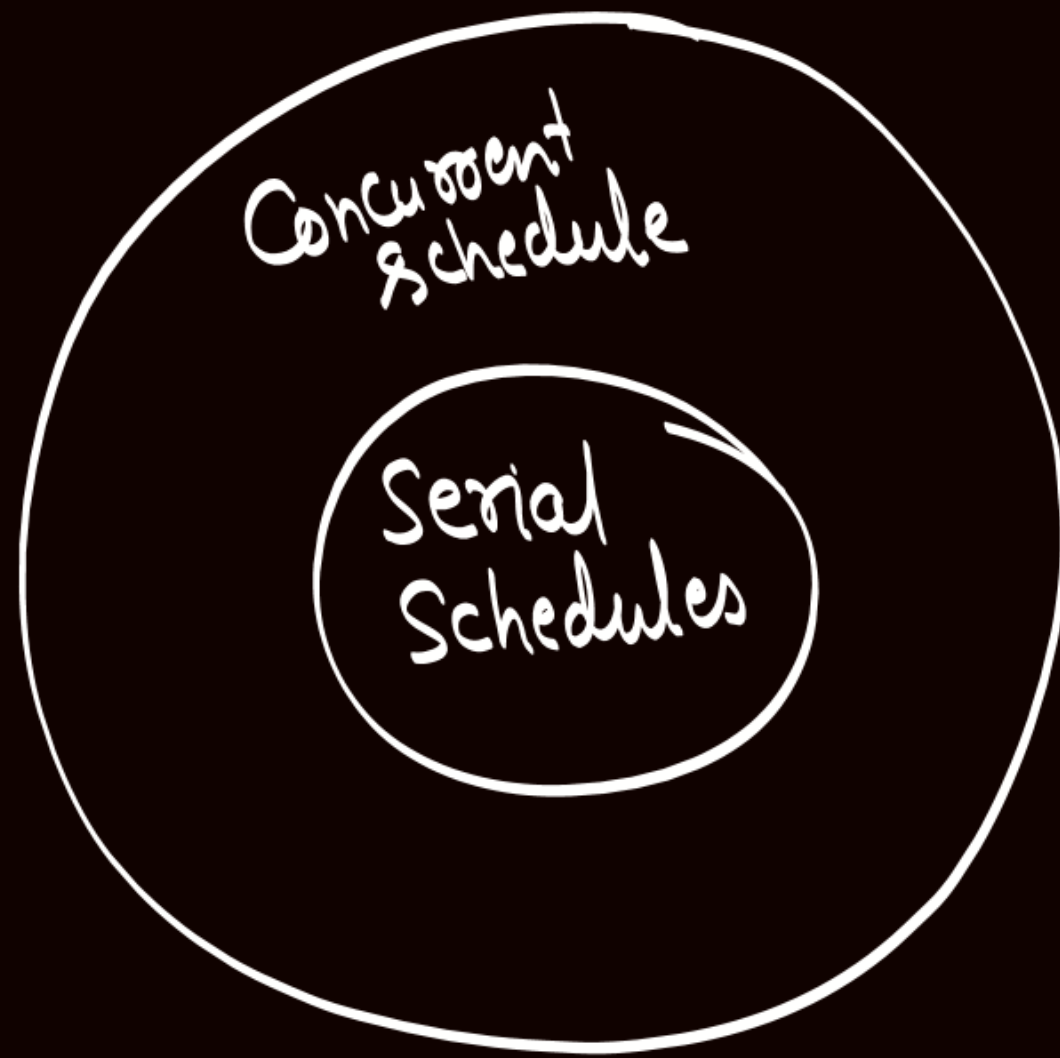
Q: Let T_1 & T_2 are two transactions with 'm' and 'n' operations respectively, How many total Schedules are possible over the transactions T_1 & T_2

$$\begin{aligned}
 \text{Total no. of Schedules possible} &= \binom{m+n}{C_m * 1} * \binom{n}{C_n * 1} = \binom{m+n}{C_m} \\
 &= \binom{m+n}{C_n * 1} * \binom{m}{C_m * 1} = \binom{m+n}{C_n} \quad (or) \\
 &\quad || \leftarrow \{n, n-1, \dots, 1\} \leftarrow \{m, m-1, \dots, 1\}
 \end{aligned}$$

Q: Let T_1, T_2 & T_3 are three transactions with 'm', 'n' and 'p' operations respectively. How many total schedules are possible over the transactions T_1, T_2 & T_3

$$\text{Total no. of schedules} = \binom{m+n+p}{m} * \binom{n+p}{n} * \binom{p}{p} \equiv \frac{(m+n+p)!}{m! n! p!}$$

$$\begin{aligned} & \binom{m+n+p}{n} * \binom{n+p}{p} * \binom{p}{p} \\ & \vdots \end{aligned}$$



All serial schedules
also included in all
Concurrent Schedules



Topic : Equivalent schedule



For two schedules to be Equivalent. following two conditions must satisfy.

① Every corresponding read operation in both the schedule must be identical.

And ② Final Update (Final write opⁿ) for every corresponding data item must be identical in both the schedules

eg. $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

$T_2: R_2(A), R_2(B)$

Consider two schedules S_1 & S_2 over the transactions T_1 & T_2

S_1

T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
	$R_2(A)$
$W_1(B)$	
	$R_2(B)$

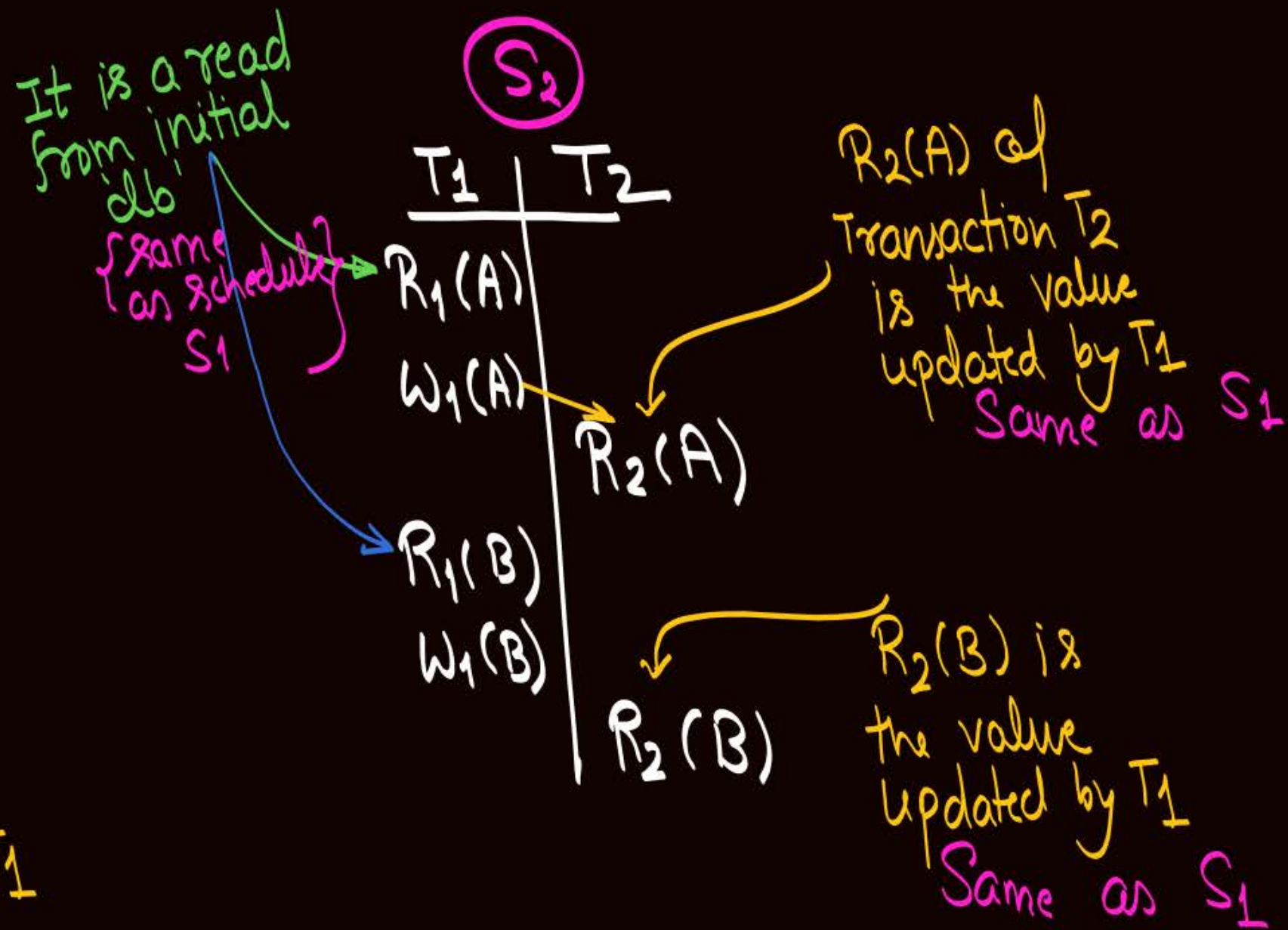
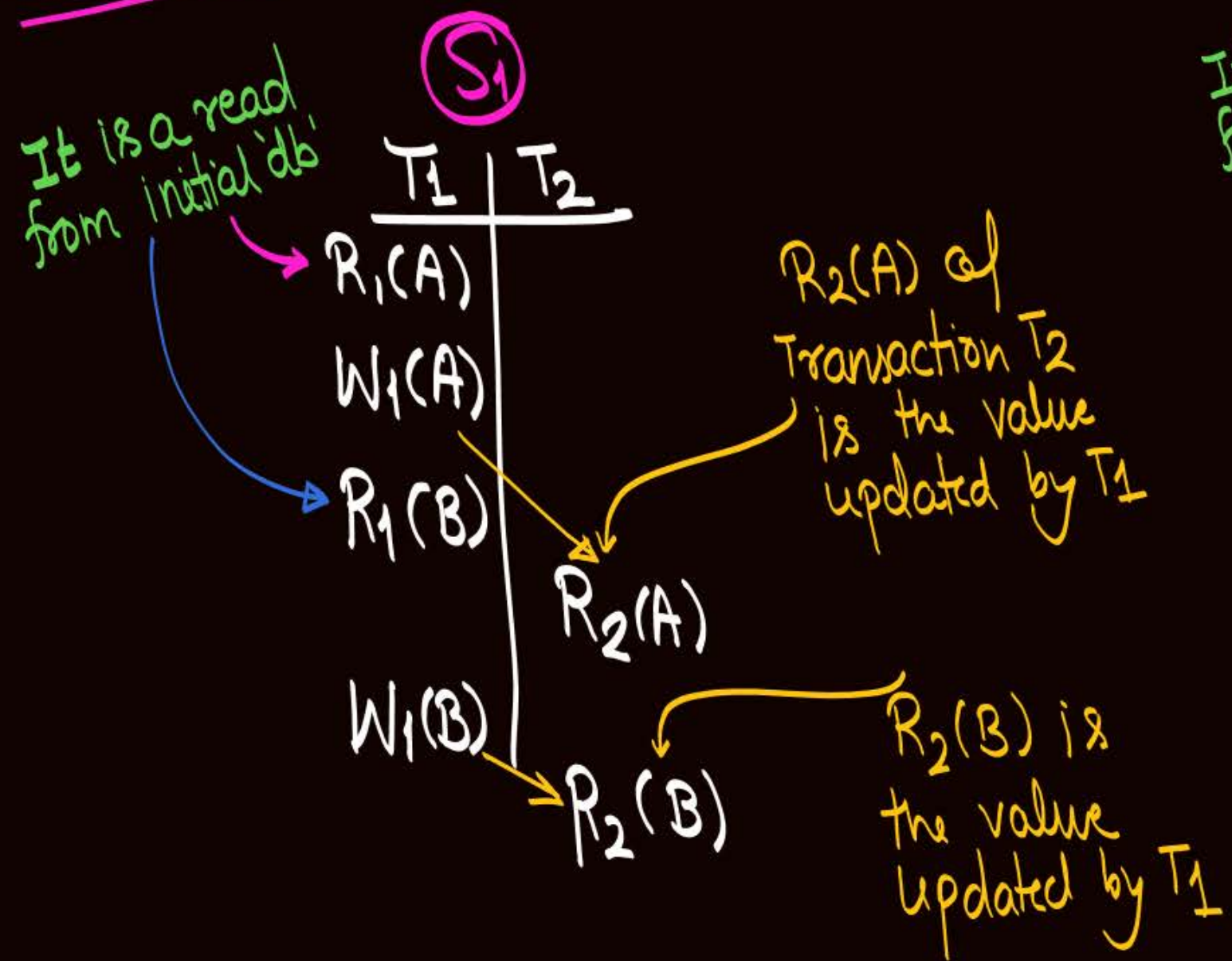
S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

Soln on
next slide

Check whether S_1 & S_2 are equivalent or not

Writ Read



Note: All read opⁿ are exactly same in both the schedules

W.r.t. Write

It is the last (final) write of data item A in schedule S1

(S ₁)	
T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
R ₁ (B)	
	R ₂ (A)
W ₁ (B)	
	R ₂ (B)

Transaction T₁ finally updates the data item A in S₁

Transaction T₁ finally updates the data item B in S₁

It is the last (final) write of data item B in schedule S1

(S ₂)	
T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
	R ₂ (A)
R ₁ (B)	
W ₁ (B)	
	R ₂ (B)

A is finally updated by T₁ in S₂ as well (same as S₁)

B is finally updated by T₁ in S₂ as well (Same as S₁)

Note: Final updates of corresponding dataitems is also identical in both the schedules.

eg. $T_1: R_1(A), W_1(A), R_1(B), W_1(B)$

$T_2: R_2(A), R_2(B)$

Consider two schedules S_1 & S_2 over the transactions T_1 & T_2

(S_1)

T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
	$R_2(A)$
$W_1(B)$	
	$R_2(B)$

(S_2)

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

Note we observed

- ① Every read is identical in S_1 & S_2
- and ② Every final update is also identical in S_1 & S_2

∴ S_1 and S_2 are equivalent

i.e. $S_1 \equiv S_2$

Check whether S_1 & S_2 are equivalent or not



Topic : Serializable schedule

{ A schedule is said to be serializable if and only if its behaviour is equivalent to at least one of the serial schedule.

Q:- Check whether the schedule is a serializable schedule or not?

①

S₁

T ₁	T ₂
from initial DB	R ₂ (A)
W ₁ (A)	
W ₁ (B)	
It is the value updated by T ₁	R ₂ (B)

Serial Schedules
writ. transactions
of schedule S₁

T₁ → T₂

T ₁	T ₂
W ₁ (A)	
W ₁ (B)	

R₂(A) ← value updated by T₁
R₂(B) ← it is not same as schedule S₁
∴ S₁ ≠ T₁ → T₂

T₂ → T₁

T ₁	T ₂
	R ₂ (A)
W ₁ (A)	
W ₁ (B)	

R₂(A) ← from initial DB same as S₁
R₂(B) ← it is from initial DB, Not same as S₁
∴ S₁ ≠ T₂ → T₁

Schedule S₁ is not equivalent to any of the serial schedules over the transactions of S₁,
∴ "S₁" is a Non-serializable schedule

Q:- Check whether the schedule is a serializable schedule or not?

②

S_2

T_1	T_2
$R_1(A)$	
	$R_2(B)$
$R_1(C)$	
	$W_2(C)$

All are from initial DB

'C' is finally updated by T_2

$T_2 \rightarrow T_1$

T_1	T_2
	$R_2(B)$
	$W_2(C)$

Value updated by T_2 $\rightarrow R_1(C)$
(Not same as S_2)
 $\therefore S_2 \neq T_2 \rightarrow T_1$

$T_1 \rightarrow T_2$

T_1	T_2
$R_1(A)$	
$R_1(C)$	
	$R_2(B)$
	$W_2(C)$

All are from initial DB (same as S_2)

'C' is finally updated by T_2 (same as S_2)

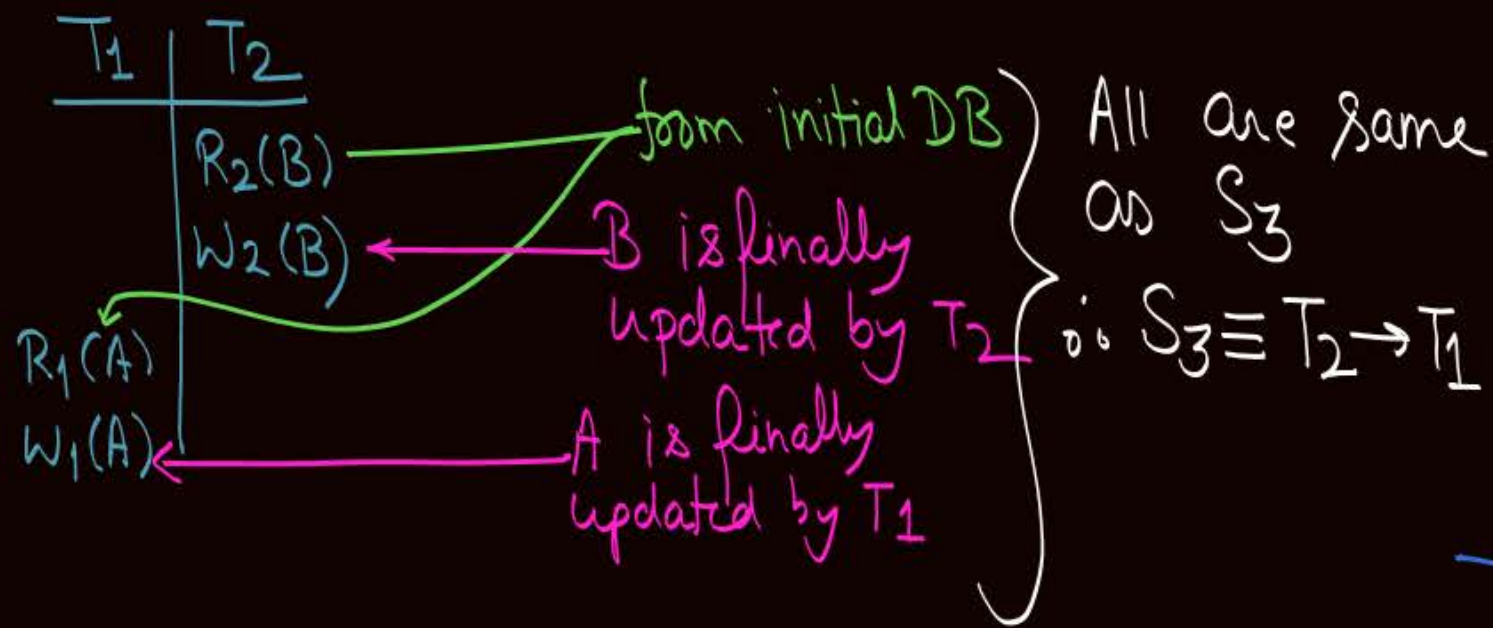
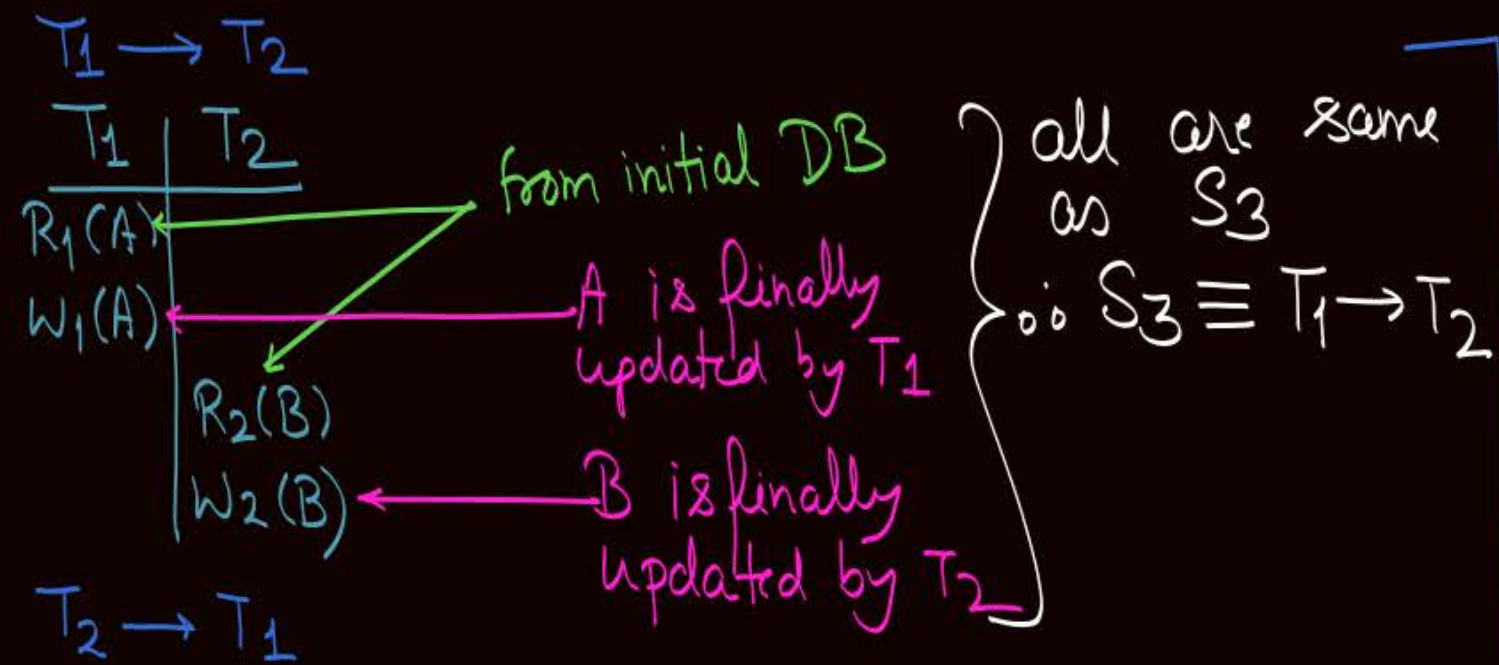
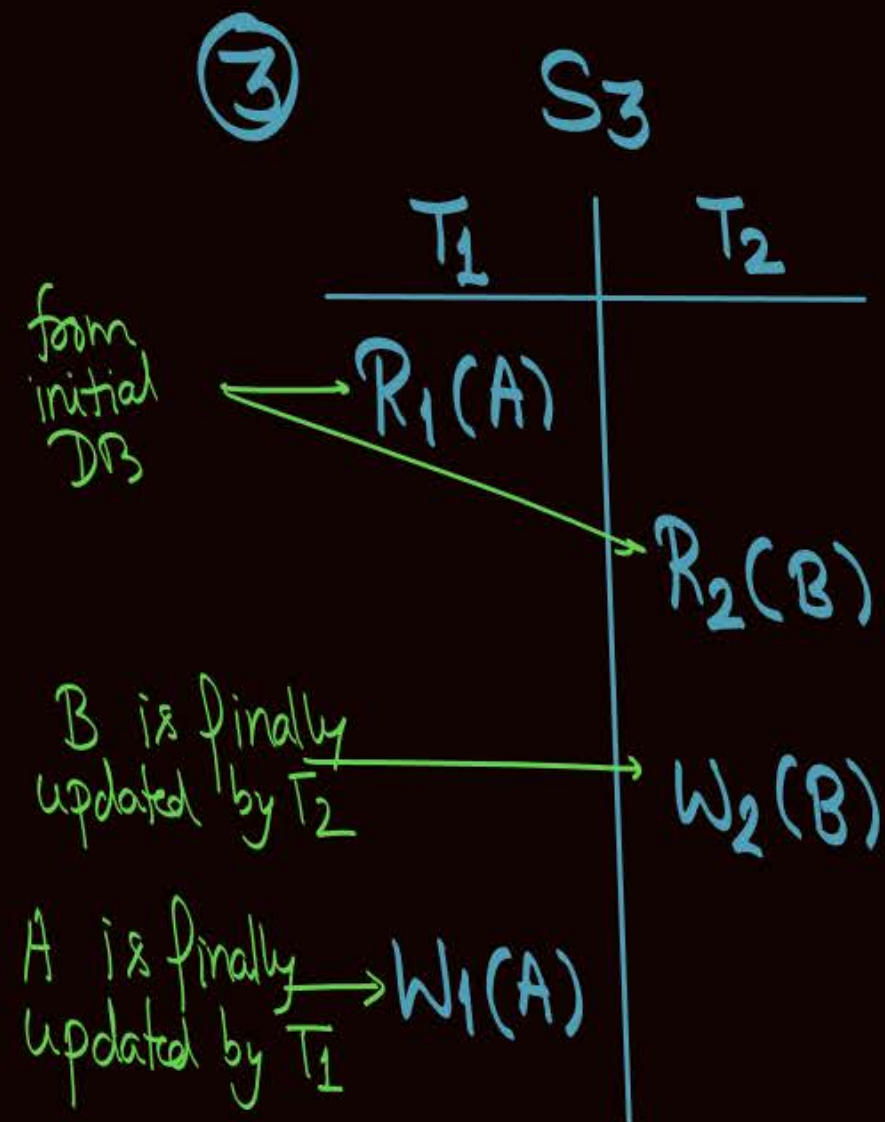
Every Read & Every final update is same

$\therefore S_2 \equiv T_1 \rightarrow T_2$

$S_2 \neq T_2 \rightarrow T_1$, but $S_2 \equiv T_1 \rightarrow T_2$

' S_2 ' is equivalent to at least one of the serial schedules
 $\therefore S_2$ is a serializable schedule, and equivalent serial schedule is $T_1 \rightarrow T_2$

Q:- Check whether the schedule is a serializable schedule or not?



Schedule S_3 is equivalent to at least one of the serial schedules
 $\therefore S_3$ is a serializable schedule, and equivalent serial schedules are $T_1 \rightarrow T_2$ & $T_2 \rightarrow T_1$

H.W.

Q:- Check whether the schedule is a serializable schedule or not?

④

S₄

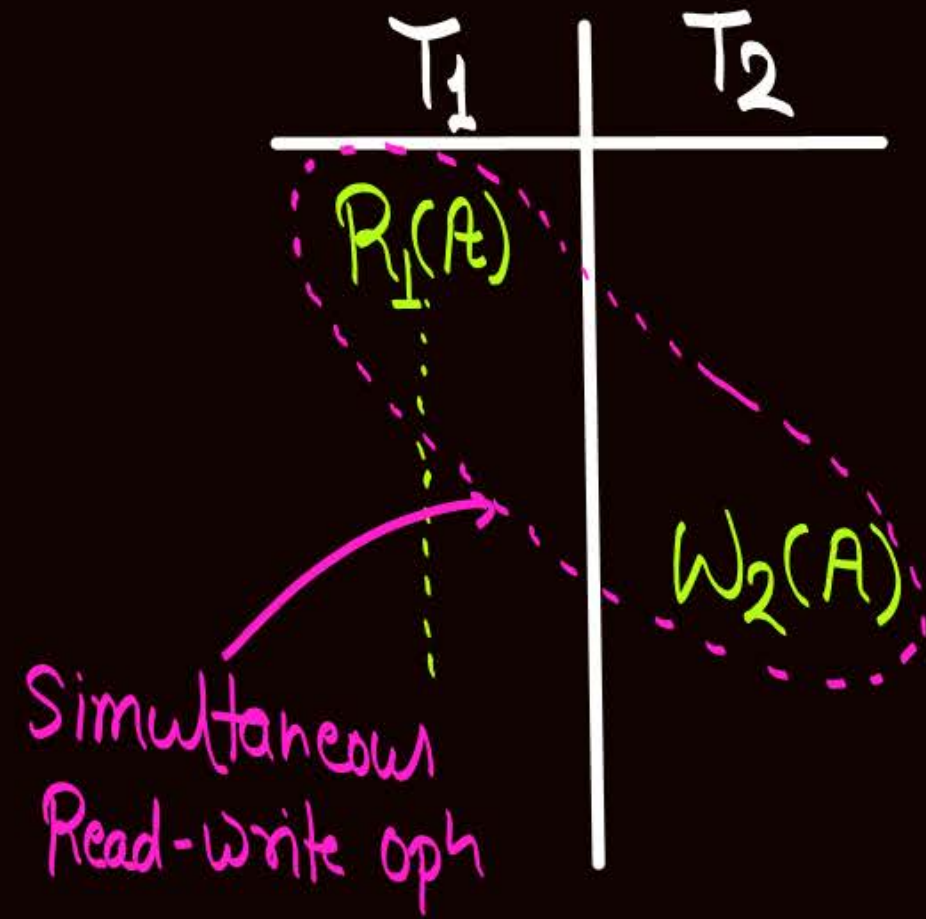
T ₁	T ₂
R ₁ (A)	
	R ₂ (B)
W ₁ (B)	
	W ₂ (B)



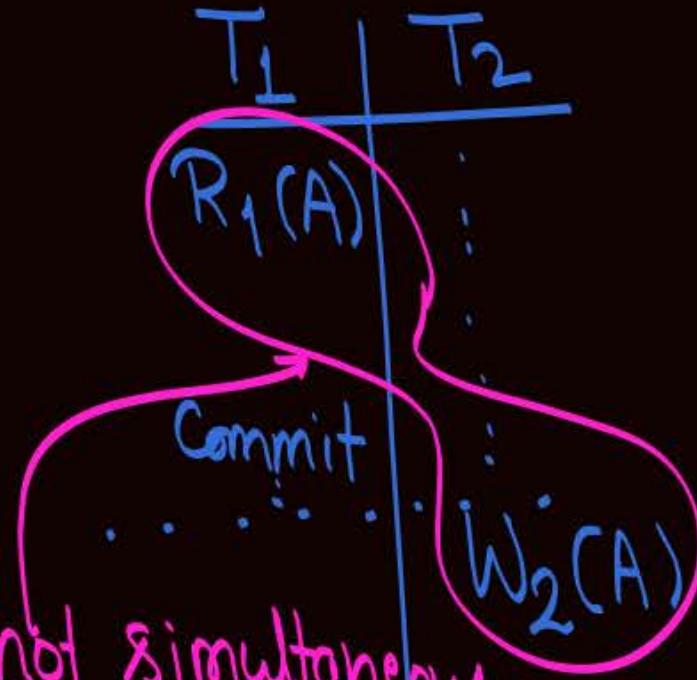
Topic : Problems because of concurrent execution

- ① R-W Problem / Read-Write Problem / Write after Read Problem
 - ② W-R Problem / Write-Read Problem / Read after Write problem
 - ③ W-W Problem / Write-Write Problem / Write after Write Problem
- } This problems exists only if the schedule is a non-serializable schedule.
-
- ④ Lost update problem
- } It is possible with both serializable as well as non-serializable schedules

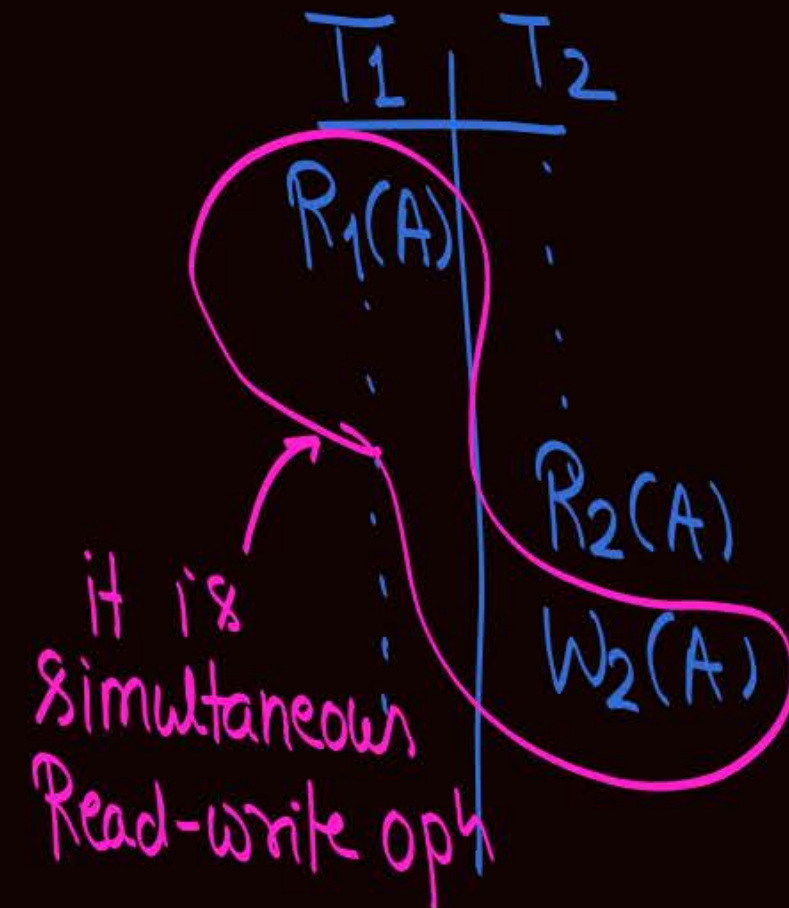
RW Problem / Read-Write Problem / Write-after-Read Problem



→ If transaction T₂ updates the data item 'A' which is already read by an uncommitted transaction T₁, then such sequence of opⁿ is called **Simultaneous Read-write opⁿ**



it is not simultaneous Read-write opⁿ because T₁ has already Committed

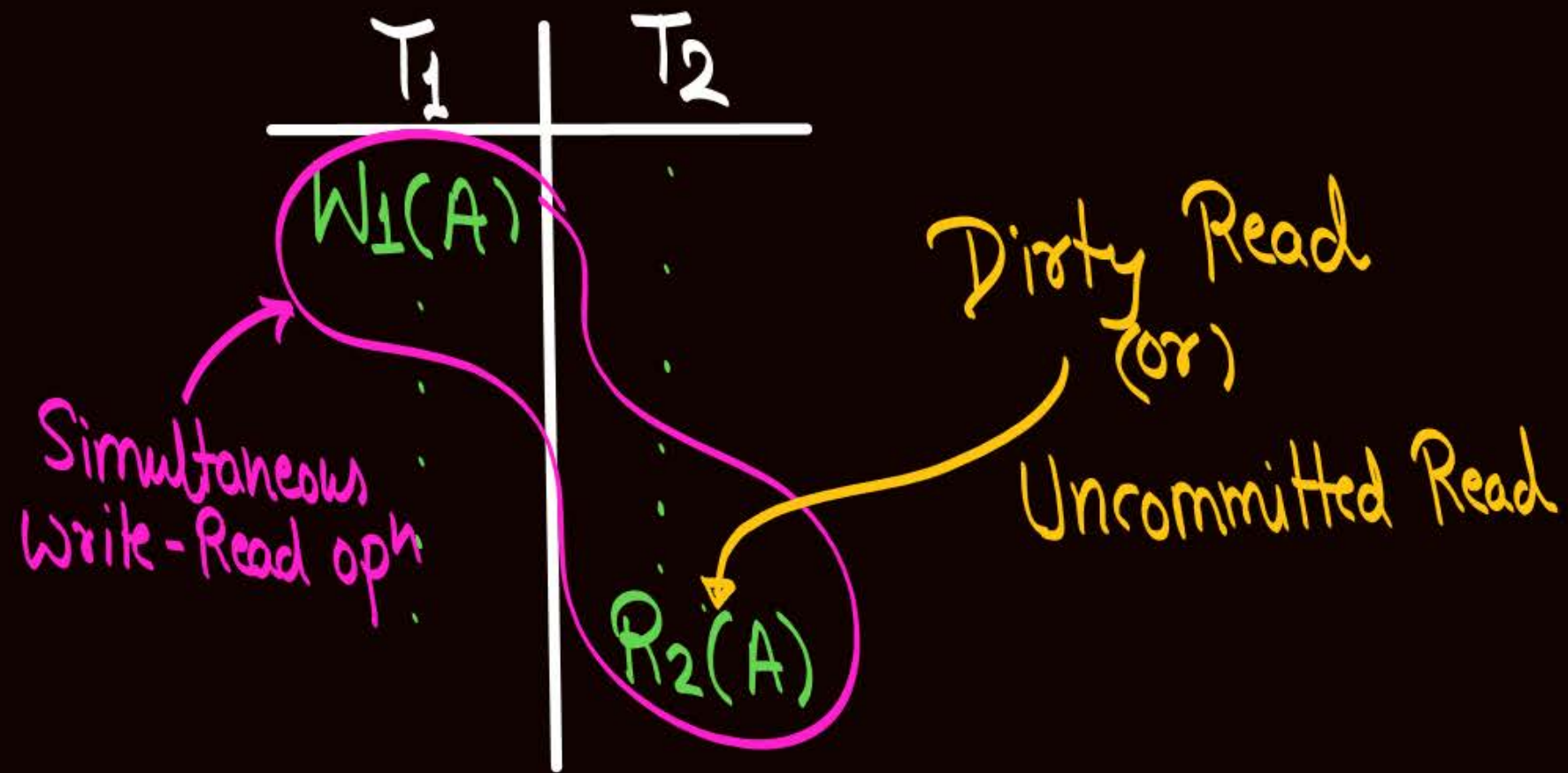


RW Problem / Read-Write Problem / Write-after-Read Problem

→ A schedule is said to have Read-Write problem if and only if

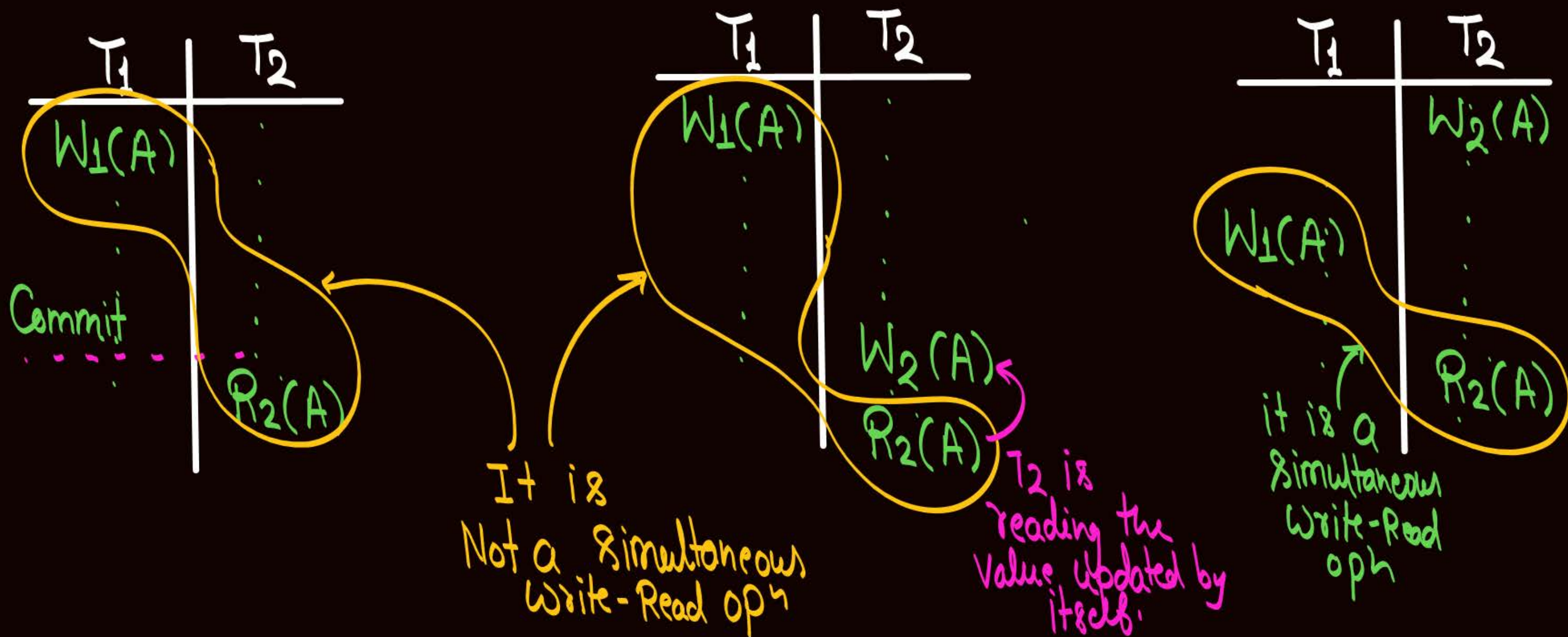
- ① Simultaneous Read-write opⁿ must exist
- and ② Schedule must be a non-serializable schedule

WR Problem / Write-Read Problem / Read-after-write Problem
also known as "Dirty Read Problem"



If transaction T_2 reads the value updated by an uncommitted transaction T_1 , then that sequence of opⁿ is called a Simultaneous Write-Read opⁿ. And, that read opⁿ is called an uncommitted read.

WR Problem / Write-Read Problem / Read-after-write Problem
also known as "Dirty Read Problem"



WR Problem / Write-Read Problem / Read-after-write Problem
also known as "Dirty Read Problem"

* A schedule is said to have Write-Read problem if and only if

① Simultaneous Write-Read opⁿ must exist.
{i.e; Uncommitted Read opⁿ must exist}

And ② Schedule must be a non-serializable schedule

H.W.

Q:- Check whether the schedule is a serializable schedule or not?

④

S₄

T ₁	T ₂
R ₁ (A)	
	R ₂ (B)
W ₁ (B)	
	W ₂ (B)



2 mins Summary



Topic

Equivalent schedule

Topic

Serializable schedule

Topic

RW problem

Topic

WR problem

Topic

WW problem

Topic

Lost update problem

Next
Class.

THANK - YOU